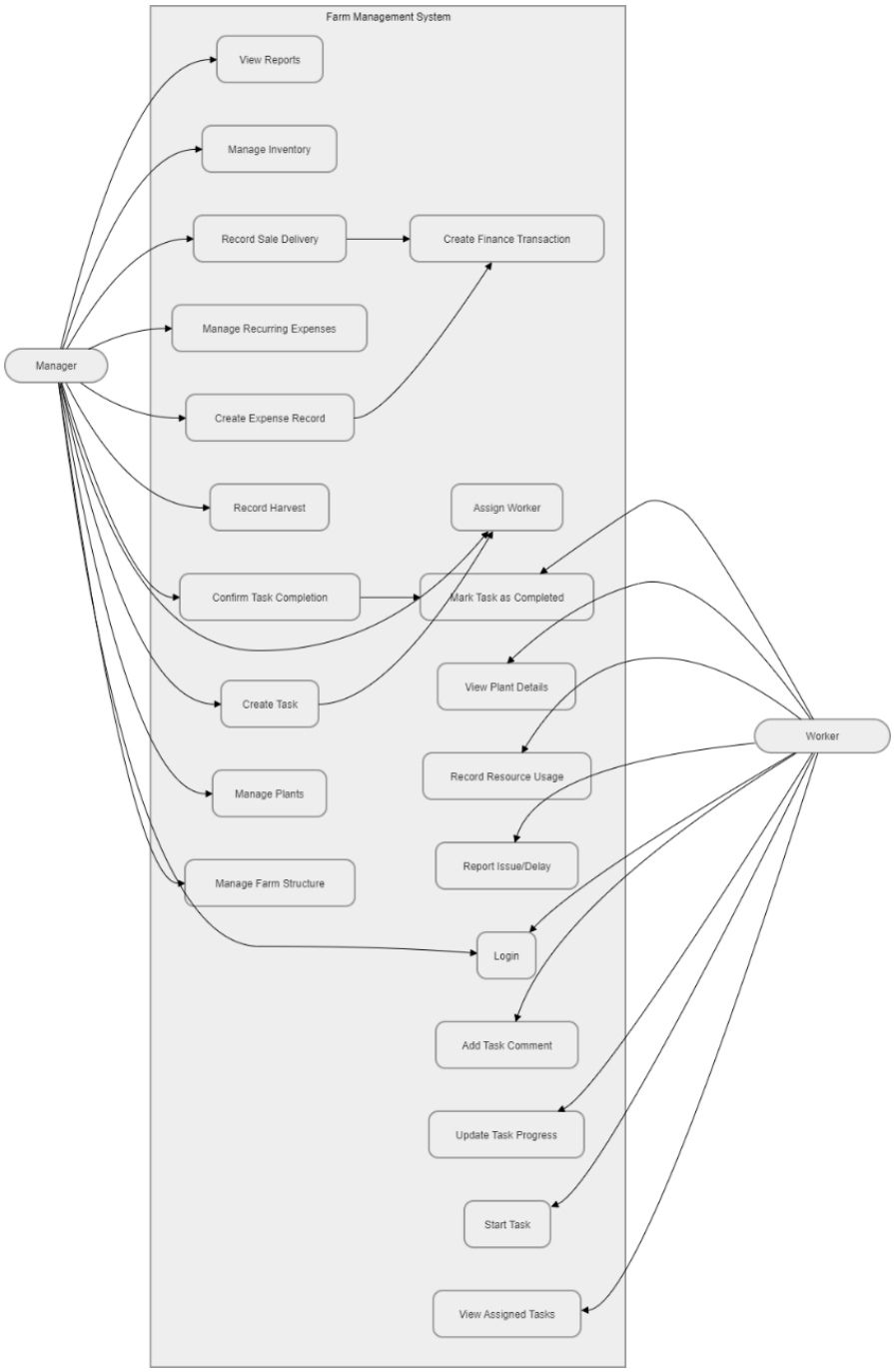
UML Behavioral Suite

1. Refined Use Case Diagram



1.1 Overview

The refined Use Case Diagram presents the main interactions between the users of the Farm Management System and the system functions. The diagram uses only the actors that directly interact with the application: Manager and Worker. This keeps the model aligned with the real system scope and with the database design, where user roles are limited to manager and worker.

1.2 Changes Made After CW3 Feedback

The Use Case Diagram was updated after feedback in order to improve correctness, clarity, and consistency with the implemented system.

The following changes were made:

- **Removed unsupported actors**
  Earlier versions risked treating external business records such as buyers or suppliers as system actors. These were removed because they do not use the application directly. Their information is entered manually by the manager and stored internally in the system.
- **Kept only the real user actors**
  The final diagram uses only:
  - **Manager**
  - **Worker**
- **Improved use case naming**
  Use case names were simplified and rewritten as clear action-based labels so that the diagram is easier to read and more consistent.
- **Separated major management functions more clearly**
  Instead of grouping too much under one vague action, the refined version keeps functions such as Manage Plants, Create Task, Assign Worker, Manage Inventory, and View Reports distinct.
- **Added proper <<include>> and <<extend>> relationships**
  These were added so the diagram satisfies UML requirements and better reflects how the system behaves.
  - Create Task <<include>> Assign Worker
  - Create Expense Record <<include>> Create Finance Transaction
  - Record Sale Delivery <<include>> Create Finance Transaction
  - Confirm Task Completion <<extend>> Mark Task as Completed

- **Aligned the diagram with the actual system modules**
  The final use cases now match the real project structure, including plant management, task management, harvest recording, finance, inventory, and reporting.

1.3 Final Actors

The final actors used in the refined Use Case Diagram are:

- **Manager**
- **Worker**

1.4 Final Use Cases

The refined Use Case Diagram includes the following use cases.

Manager use cases

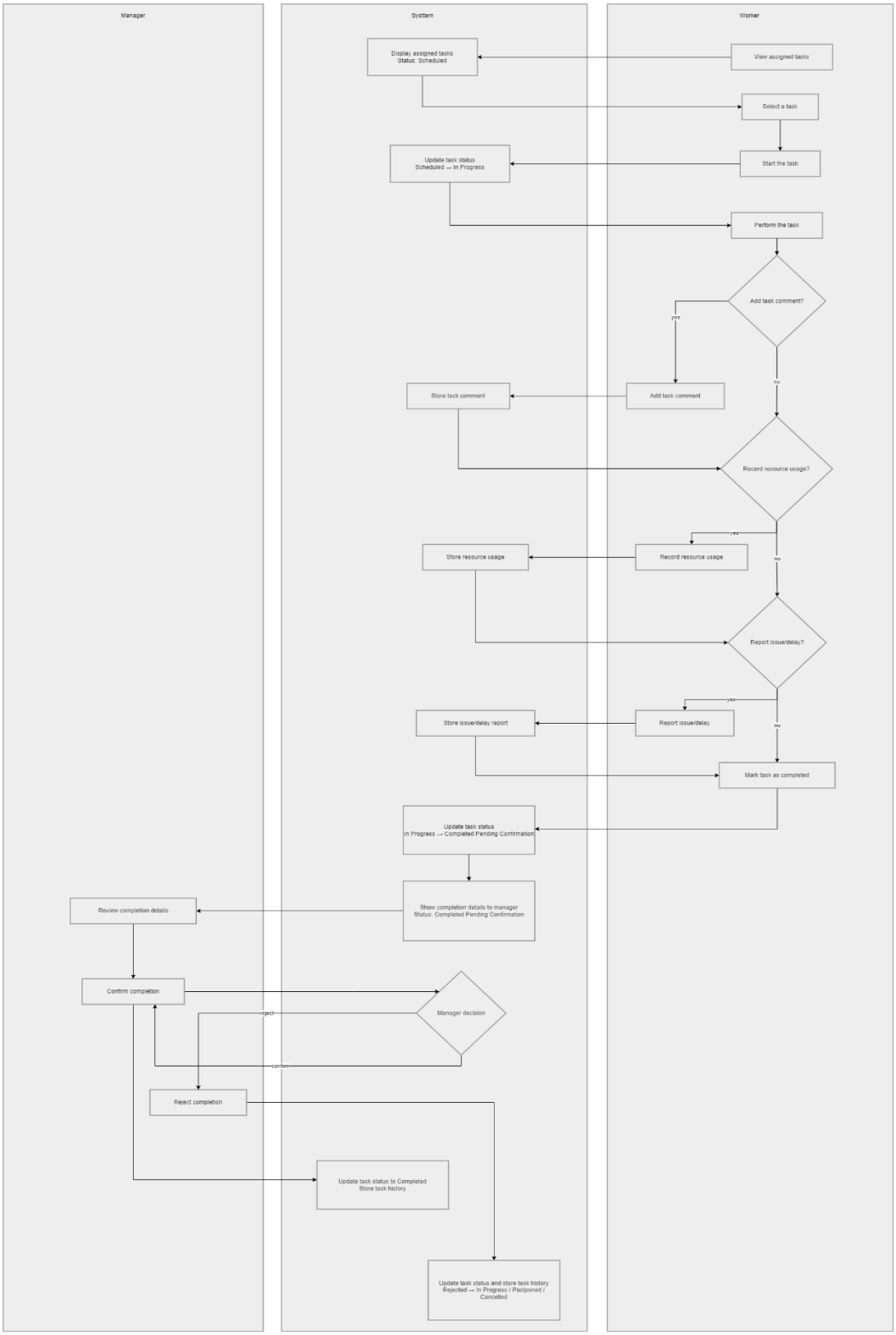
- Log In
- Manage Farms
- Manage Plots
- Manage Greenhouses
- Manage Plants
- Create Task
- Assign Worker
- Postpone Task
- Cancel Task
- Confirm Task Completion
- Record Harvest
- Create Expense Record
- Create Finance Transaction
- Manage Recurring Expenses
- Create Sales Deal
- Record Sale Delivery

- Manage Inventory
- View Reports

Worker use cases

- Log In
- View Assigned Tasks
- Start Task
- Update Task Progress
- Add Task Comment
- Report Issue/Delay
- Record Resource Usage
- Mark Task as Completed
- View Plant Details

2. Activity Diagram with Swimlanes



2.1 Chosen Workflow

The Activity Diagram models the workflow:

Update Task Progress and Confirm Completion

This was selected because it is a key multi-actor process in the system and clearly involves interaction between both the Worker and the Manager. It also reflects the real task lifecycle supported by the database, including task progress, worker completion, manager confirmation, and task history/status updates.

2.2 Swimlanes Used

The Activity Diagram uses swimlanes to separate responsibilities clearly. The swimlanes are:

- **Worker**
- **System**
- **Manager**

2.3 Main Behaviour Shown

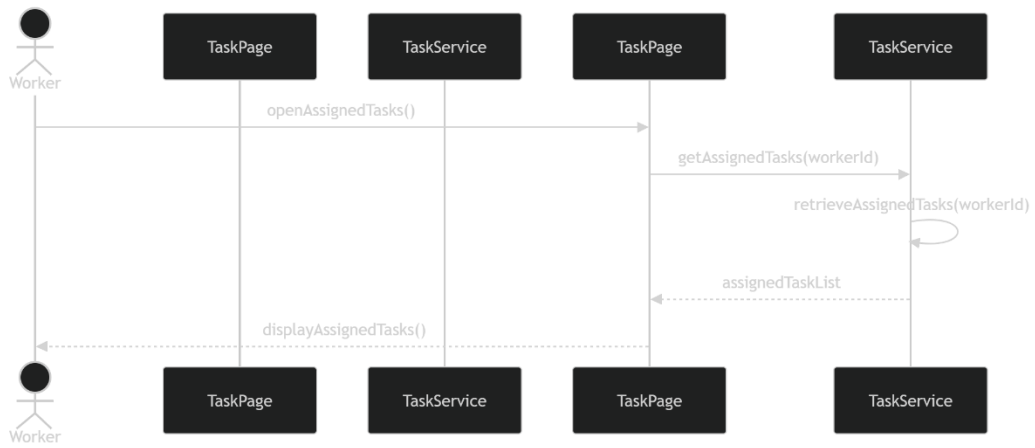
The diagram shows the following general flow:

- worker opens assigned tasks
- worker selects a task
- worker starts the task
- system updates task status
- worker performs the work
- worker may add notes, comments, or resource usage
- worker marks the task as completed
- system changes the task to pending confirmation
- manager reviews the completion
- manager either confirms completion or rejects it
- system updates the final task status

2.4 Why This Workflow Was Chosen

This workflow was chosen because it is one of the most important operational processes in the application. It also makes swimlanes meaningful, since responsibility moves between multiple actors and the system. It is more suitable than a single-actor workflow because the purpose of swimlanes is to show who performs each activity.

3. Sequence Diagram 1 – Simple Use Case



3.1 Chosen Use Case

The simple sequence diagram models:

View Assigned Tasks

3.2 Why It Is Considered Simple

This use case is suitable for the simple sequence diagram because it has:

- a short interaction flow
- limited logic
- only a small number of participating objects

It can be modelled clearly with **2 to 3 objects**.

3.3 Main Objects Used

The diagram should remain simple and use a small set of objects, such as:

- **Worker**
- **Task Page / Task Interface**
- **Task Service or Task Repository**

3.4 Main Behaviour Shown

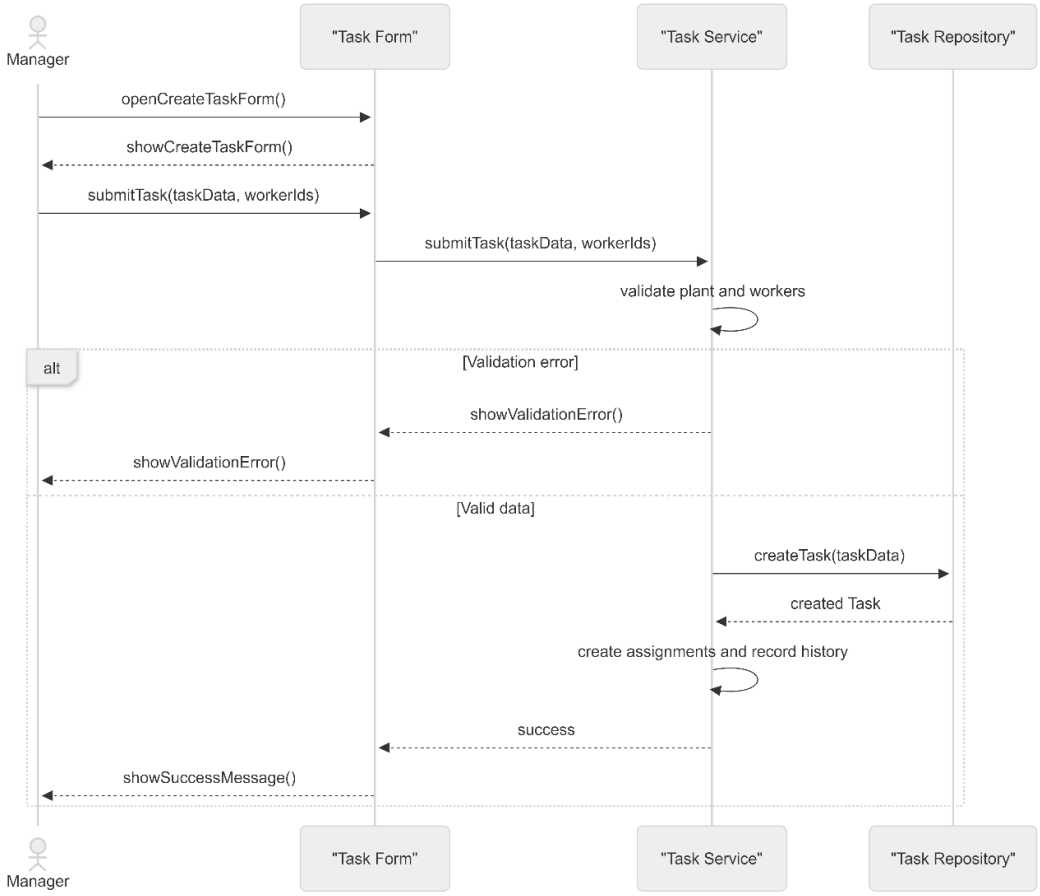
The diagram shows the worker requesting their assigned task list, the system retrieving the relevant task records, and the task list being returned to the worker interface.

3.5 Consistency Note

The actor name **Worker** and the task-related naming used in this diagram must remain identical to the names used in the Use Case Diagram and the Activity Diagram.

---

4. Sequence Diagram 2 – Complex Use Case



4.1 Chosen Use Case

The complex sequence diagram models:

Create Task and Assign Worker

4.2 Why It Is Considered Complex

This use case is suitable for the complex sequence diagram because it involves:

- more than four participating objects
- validation steps
- conditional behaviour
- multiple related records being created
- interaction between task creation and worker assignment

This makes it a much better example of a complex system interaction than simpler read-only actions.

4.3 Main Objects Used

The diagram can include objects such as:

- **Manager**
- **Task Form / Task Page**
- **Task Controller / Task Service**
- **Plant Repository**
- **Worker Repository**
- **Task Repository**
- **Task Assignment Repository**
- **Task History Repository**

4.4 Main Behaviour Shown

The diagram shows the following type of flow:

- manager opens task creation
- system loads required reference data
- manager enters task details

- system validates the selected plant
- system validates selected worker(s)
- system creates the task
- system creates one or more assignment records
- system stores task history
- system returns success confirmation

4.5 Required Complexity

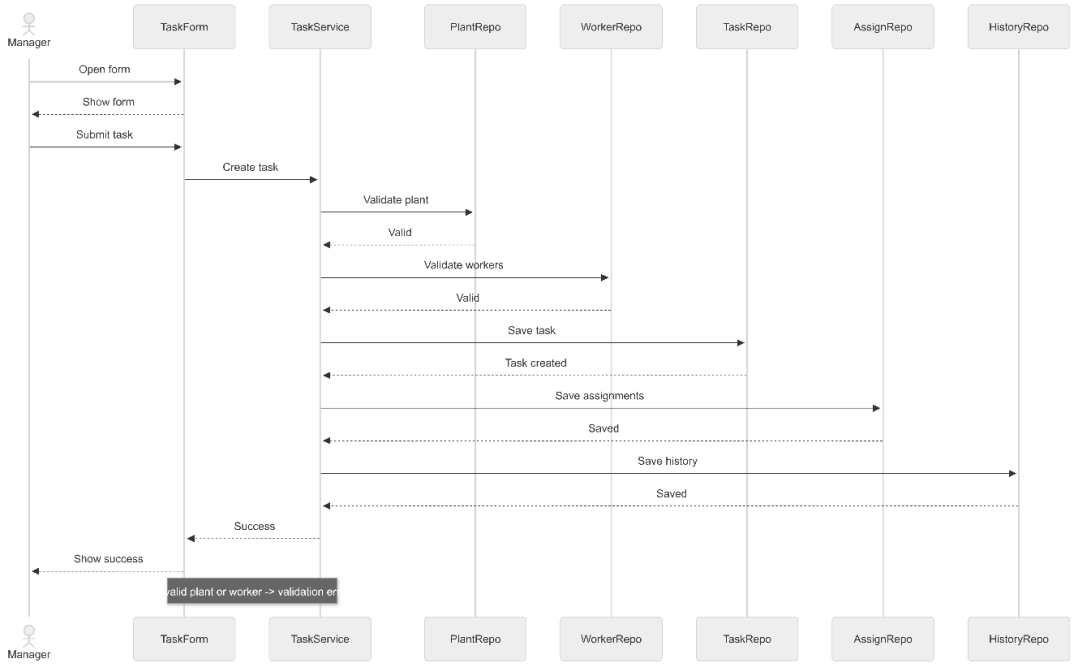
This sequence diagram is the complex one because it should include:

- at least 4 objects
- at least one condition such as invalid worker or invalid plant
- optionally a loop where assignment is repeated for multiple workers

4.6 Consistency Note

All object and actor names must match the naming already used elsewhere in the report, especially Manager, Worker, Task, Task Assignment, and Plant.

5. Communication Diagram



5.1 Source of the Diagram

The Communication Diagram is converted from:

Sequence Diagram 2 – Create Task and Assign Worker

5.2 Purpose

The purpose of the Communication Diagram is to represent the same interaction as the chosen sequence diagram, but in a structure-focused way. Instead of emphasizing time order vertically, it emphasizes how the participating objects are linked and how messages pass between them.

5.3 Main Participants

The Communication Diagram uses the same participants as the selected sequence diagram so that both diagrams stay internally consistent.

Suggested participants:

- **Manager**
- **Task Form / Task Page**
- **Task Controller / Task Service**
- **Plant Repository**
- **Worker Repository**
- **Task Repository**
- **Task Assignment Repository**
- **Task History Repository**

5.4 Behaviour Represented

The diagram should show the same logic as the complex sequence diagram:

- validation of inputs
- creation of task record
- creation of assignment record(s)
- storage of task history
- return of success result

5.5 Consistency Note

Because this diagram is converted from the complex sequence diagram, the names, participants, and message meaning must remain the same. Only the visual style changes.

6. Internal Consistency Across All Diagrams

All diagrams in this section use the same names for actors, objects, and system elements so that the modelling remains internally consistent.

The names that must remain the same across all diagrams include:

- **Manager**
- **Worker**
- **Plant**
- **Task**
- **Task Assignment**
- **Task History**
- **Resource Usage**
- **Expense Record**
- **Finance Transaction**
- **Inventory Item**

This consistency is important because all diagrams describe behaviour within the same Farm Management System and are based on the same underlying project structure.

7. Design Rationale

The selected diagrams were chosen to model the most important behaviours of the Farm Management System while keeping the set balanced and meaningful.

The refined Use Case Diagram was included because it gives the overall behavioural view of the system and shows the responsibilities of the two real actors, Manager and Worker. It was updated from the earlier version to remove unsupported actors and to better reflect the actual implemented scope of the application.

The Activity Diagram with swimlanes focuses on Update Task Progress and Confirm Completion because this is a key multi-actor workflow. It clearly shows how responsibility moves between the worker, the system, and the manager. This makes it an ideal candidate for swimlane modelling.

The simple Sequence Diagram uses View Assigned Tasks because it is a straightforward interaction with a small number of objects. This helps demonstrate a clear and readable sequence diagram for a basic use case.

The complex Sequence Diagram uses Create Task and Assign Worker because it includes validation, multiple related objects, conditional behaviour, and the creation of linked records. This makes it suitable for showing a more realistic and detailed internal interaction.

The Communication Diagram was derived from the complex sequence diagram because that use case already contains a rich set of interacting objects. Converting it into a communication diagram makes it easier to show the structural relationships between the participants while keeping the same behaviour.

Overall, these use cases were chosen because they represent the core operational behaviour of the system: task viewing, task execution, task management, and coordination between manager and worker. Together, they provide a balanced view of both simple and complex behaviour in the application.